

Ride-Sharing Analytics Platform

Sec - 04

TEAM NO : 20

2420090101 – N DhanuSree

2420090032 - Y Viditha

2420030678 – G GopiKrishna

Under the guidance of

Dr. NALLA SIRISHA

Associate Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Table of Contents

- Abstract
- Introduction
- Objectives
- Existing System
- Limitations of Existing System
- Proposed System
- Literature Survey
- Software Requirements
- Hardware Requirements
- Functional Requirements
- Non-Functional Requirements
- System architecture
- UML Diagrams
- Methodology
- Technologies Used
- Implementation
- Functionality / Flow Diagram
- Results
- Comparison
- Conclusion
- References

Abstract

The rapid growth of ride-sharing platforms like Uber has made data-driven decision-making essential for improving service efficiency and customer experience in urban transportation. This project, "Ride-Sharing Analytics Platform," proposes an Uber-like system that stores trip information, calculates peak traffic hours, tracks driver performance, and predicts future ride demand. The platform is built using Python for data processing, PostgreSQL for structured storage, Apache Kafka for real-time streaming, Apache Airflow for ETL pipeline orchestration, Docker for containerized deployment, and Power BI for interactive dashboard visualization. By automating data collection and transformation through scalable ETL pipelines, the system delivers actionable insights on demand patterns, traffic trends, and driver performance, supporting better fleet management and pricing decisions. Through this project, practical skills are gained in ETL pipeline development, database management, data warehousing, and dashboard design — key competencies for real-world data engineering roles.

Introduction

- The emergence of ride-sharing platforms has significantly reshaped urban mobility, offering passengers a convenient, technology-driven alternative to traditional modes of transportation.
- These platforms generate massive volumes of data daily — including trip requests, real-time driver locations, fare details, and user behavior patterns — making efficient data analysis critical for service improvement.
- Despite this abundance of data, many platforms lack a structured analytical framework to convert raw data into actionable business insights.
- Common operational challenges include inconsistent driver availability during peak hours, inefficient route allocation, and unpredictable fare surges, all of which affect customer satisfaction and platform reliability.
- This project addresses these challenges by developing a Ride-Sharing Analytics Platform that processes trip data through automated pipelines to generate insights on demand, traffic, and driver performance.
- The system leverages modern data engineering tools to deliver a scalable, real-time analytics solution that supports informed, data-driven decision-making

Objectives

- To design and develop a data analytics platform that stores comprehensive ride-sharing trip information (pickup/drop location, fare, duration)
- To calculate and visualize peak traffic hours across different zones for better fleet planning
- To build a module for tracking and evaluating individual driver performance
- To implement a predictive model that forecasts future ride demand using historical data
- To build automated ETL pipelines using Apache Airflow for reliable, scheduled data processing
- To enable real-time data ingestion using Apache Kafka for continuous trip event streaming
- To containerize the entire application using Docker for consistent, scalable deployment
- To present all insights through an interactive Power BI dashboard for data-driven decision-making

Existing System

- Existing ride-sharing platforms collect trip, driver, fare, and user-related data.
- Data is generally stored across different systems and databases.
- Analysis is mainly performed on previously collected trip data.
- Peak traffic and ride-demand patterns are identified from historical data.
- Driver performance is evaluated using available trip information.
- Data processing may involve separate tools for storage, processing, and visualization.
- Real-time streaming and automated ETL are not fully integrated into a single analytical workflow.

Limitations of Existing System

- **Delayed Data Analysis:** Insights may not be available immediately when new trip data arrives.
- **Demand Uncertainty:** Sudden changes in ride demand are difficult to identify.
- **Driver Availability:** Maintaining sufficient drivers during peak hours can be challenging.
- **Inefficient Fleet Planning:** Lack of timely insights can affect driver and vehicle allocation.
- **Fare Surge Challenges:** Sudden demand variations can result in unpredictable fare changes.
- **Data Silos:** Data from different sources may be difficult to integrate.
- **Limited Automation:** Manual data processing can increase processing time.
- **Scalability Issues:** Handling continuously increasing ride data can become difficult.

Proposed System

- Develop a **Ride-Sharing Analytics Platform** for centralized ride-data analysis.
- Store trip, driver, and payment data in **PostgreSQL**.
- Use **Apache Kafka** for real-time trip-event streaming.
- Use **Apache Airflow** to automate ETL pipelines.
- Use **Python** for data processing and demand prediction.
- Analyze **peak hours, traffic patterns, demand trends, and driver performance**.
- Use **Power BI** for interactive visualization and dashboards.
- Use **Docker** for containerized and scalable deployment.
- Provide data-driven insights for better fleet management and decision-making.

Literature Survey

S.No	Research paper	Author(s)	Link	Drawback
1.	Short-term prediction of demand for ride-hailing services: A deep learning approach (UberNet)	Long Chen et al.	https://arxiv.org/abs/2212.03956	Trained on NYC-specific data; may not generalize well to other cities with different traffic patterns
2.	Analysis and prediction of New York City taxi and Uber demands	(NYC study, published in scielo.org.mx)	scielo.org.mx/scielo.php?script=sci_arttext&pid=S1665-64232023000500886	Built around NYC's zip-code zones and weather data; not directly applicable to other cities
3.	Ride-hailing fares and demand interactions	(ScienceDirect, 2026)	https://sciencedirect.com/science/article/abs/pii/S0739885926000351	Focuses only on Uber and Lyft in New York City over a short 15-day window, so findings may not hold for other markets or longer time periods.
4.	Predicting rideshare demand (Public Policy Analytics, Ch. 8)	Ken Steif	https://urbanspatial.github.io/PublicPolicyAnalytics	Notes that spatio-temporal forecasting remains an open research problem, and reactive routing approaches often arrive after a demand spike has already ended.
5.	Modeling and prediction of ride-sharing utilization dynamics	(Hindawi, Journal of Advanced Transportation)	https://hindawi.com/journals/jat/2019/6125798	The prediction method is validated on a single dataset of NYC taxi trips from a specific month, limiting confidence in broader applicability.
6.	Real-time text analytics pipeline using open-source big data tools	Hassan Nazeer, Waheed Iqbal, Fawaz Bokhari, Faisal Bukhari, Shuja Ur Rehman Baig	https://arxiv.org/pdf/1712.04344	Evaluated only on Twitter sentiment data at a relatively small scale (under half a million tweets), so scalability to much larger, continuous ride-sharing data streams is not proven.

Software Requirements

Software	Purpose
Python	Data processing and demand prediction
PostgreSQL	Storing trip, driver and payment data
Apache Kafka	Real-time event streaming
Apache Airflow	ETL pipeline orchestration
Docker	Containerized deployment
Power BI	Dashboard and data visualization
Operating System	Windows / Linux

Hardware Requirements

- **Processor:** Intel Core i5 or equivalent
- **RAM:** Minimum 8 GB
- **Storage:** Minimum 256 GB SSD
- **Network:** Stable Internet connection
- **Display:** Standard monitor/display
- **System:** 64-bit computer

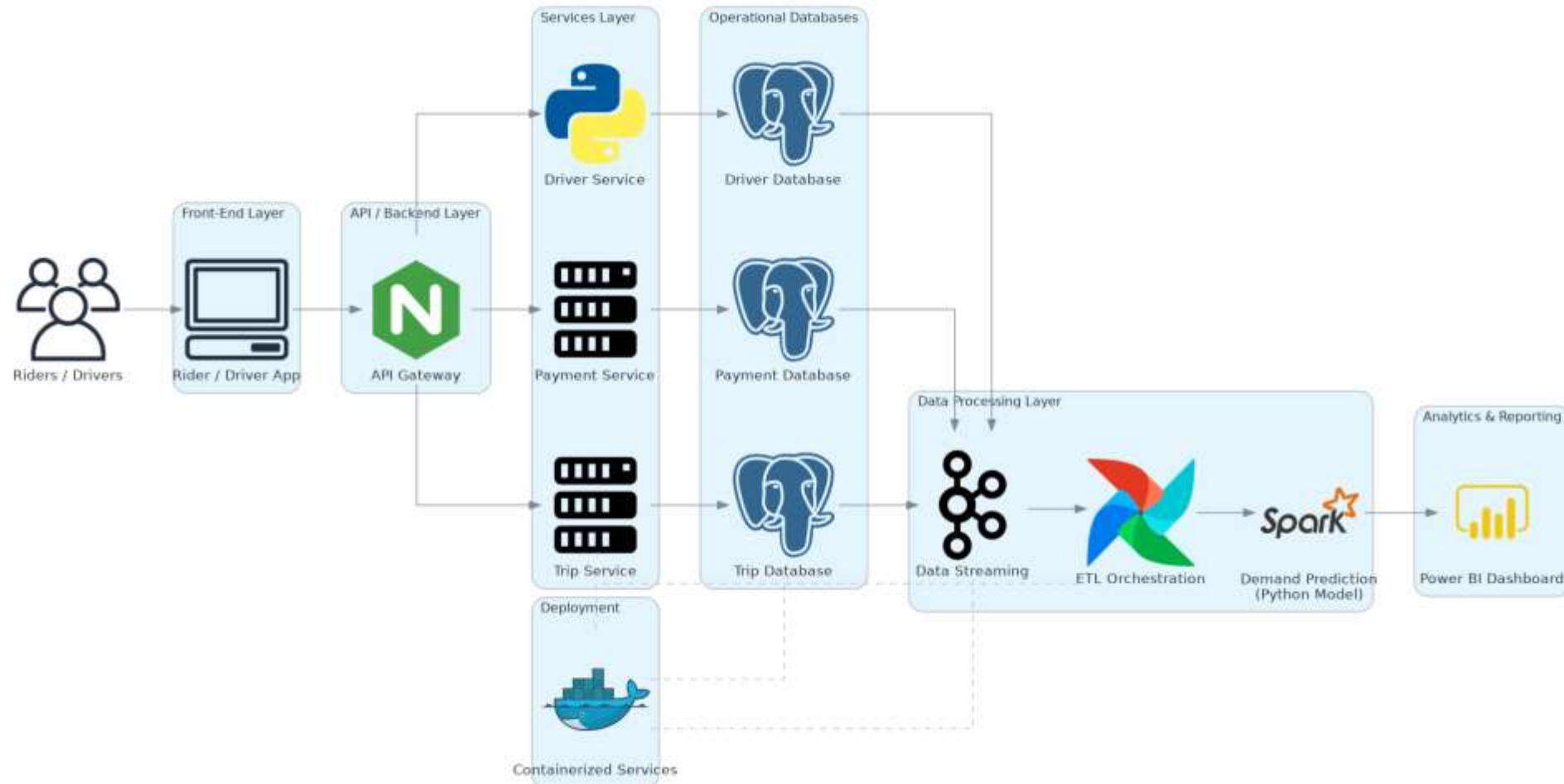
Functional Requirements

- The system should:
- **Collect Ride Data**
 - Capture trip, driver, and payment information.
- **Store Data**
 - Store structured information in PostgreSQL.
- **Stream Real-Time Events**
 - Send trip events through Apache Kafka.
- **Process Data**
 - Clean and transform data using ETL pipelines.
- **Analyze Ride Data**
 - Identify peak hours, zones, demand patterns and driver performance.
- **Predict Ride Demand**
 - Use historical data to forecast future demand.
- **Generate Dashboards**
 - Display analytical results using Power BI.
- **Support Decision-Making**
 - Provide insights for fleet management and planning.

Non-Functional Requirements

- **Scalability:** The system should handle increasing ride data volumes.
- **Performance:** Data should be processed efficiently for timely analysis.
- **Reliability:** ETL and streaming pipelines should process data consistently.
- **Availability:** The platform should be available for continuous data processing.
- **Maintainability:** Modular Python components should be easy to maintain.
- **Security:** Ride and user-related data should be stored and processed securely.
- **Usability:** Power BI dashboards should present insights clearly.
- **Portability:** Docker should allow consistent deployment across environments.

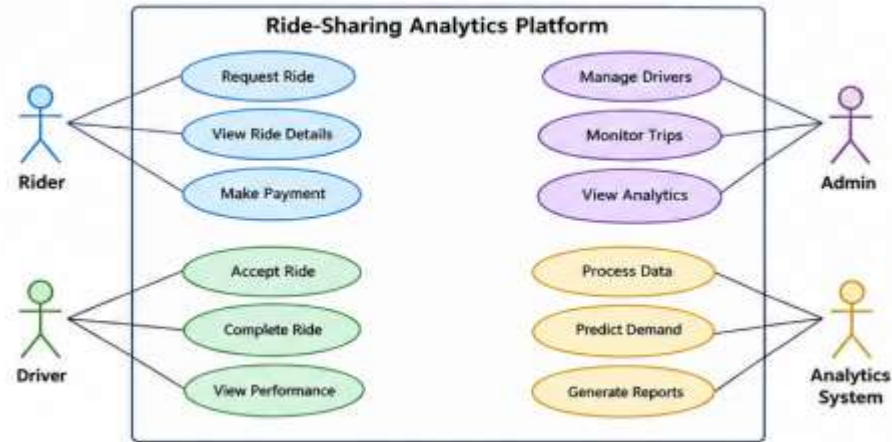
System Architecture



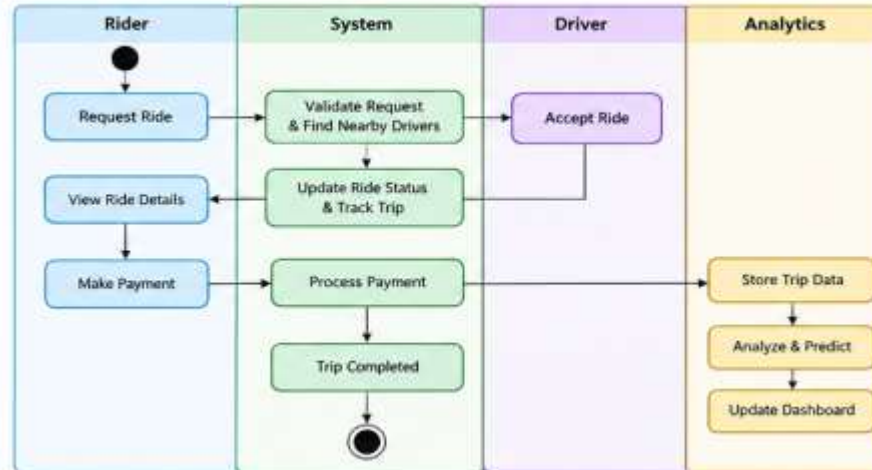
Ride-Sharing Analytics Platform Architecture

UML Diagrams

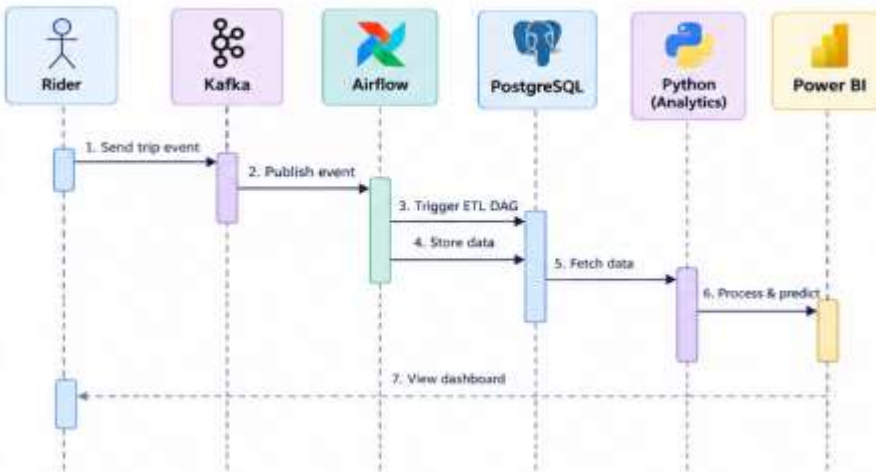
Use Case Diagram – Ride-Sharing Analytics Platform



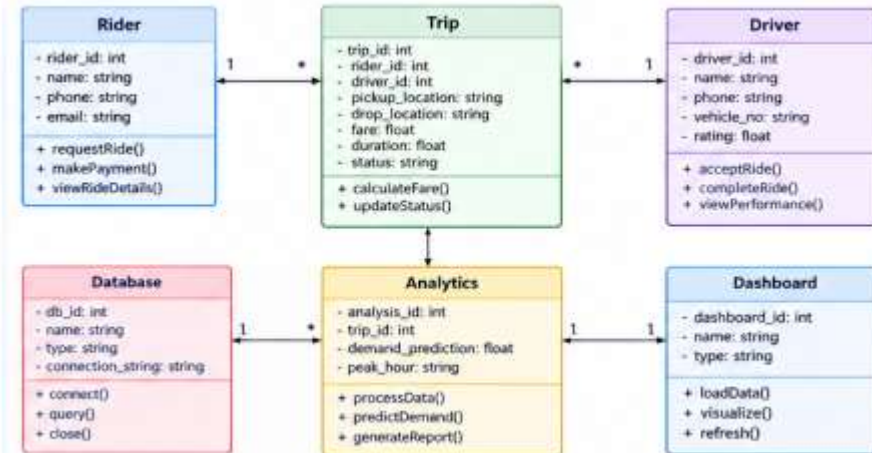
Activity Diagram – Ride Request to Analytics



Sequence Diagram – Real-time Data Flow (Kafka + Airflow)



Class Diagram – Ride-Sharing Analytics Platform



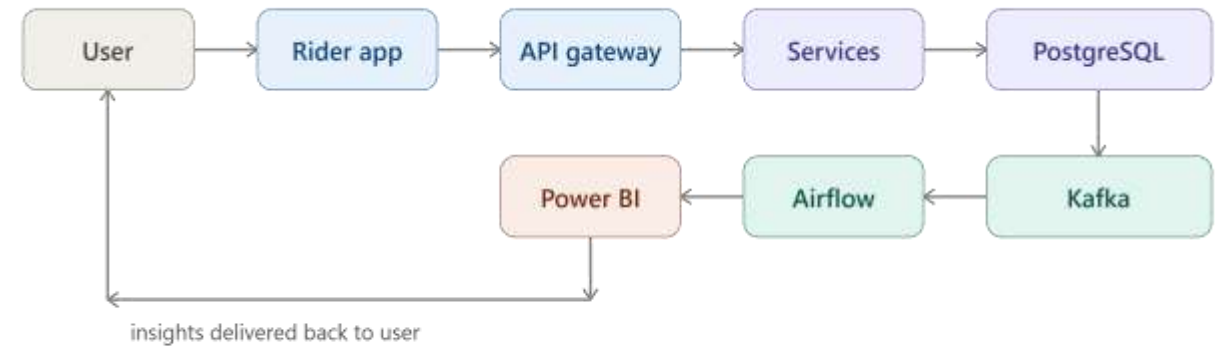
Methodology, System Design and Architecture

Methodology

- Collect trip, driver, and payment data from the app
- Stream real-time trip events using Kafka
- Clean and transform data via automated Airflow pipelines
- Analyze traffic, driver performance, and demand trends
- Display insights on Power BI dashboards

System Design

- Riders/drivers generate trip, driver, and payment events
- API Gateway routes requests to backend services
- Services store structured data in PostgreSQL
- Pipeline streams and transforms data in real time
- Analytics engine produces traffic and demand insights



Use of Modern Tools and Technologies

Technologies Used

- **Processing:** Python – data processing and demand prediction logic
- **Storage:** PostgreSQL – structured storage for trip, driver, and payment data
- **Streaming:** Apache Kafka – real-time streaming of trip events
- **Automation:** Apache Airflow – orchestration of automated ETL pipelines
- **Deployment:** Docker – containerized deployment of backend services
- **Visualization:** Power BI – interactive dashboards for insights
- These tools are widely used in real-world data engineering roles at companies like Uber, Netflix, and Amazon — reflecting industry-standard practices rather than academic-only methods.

Benefits

- Efficient, real-time data processing at scale.
- Better analysis of traffic and demand patterns.
- Supports accurate driver performance tracking.
- Helps improve fleet management and decision-making.

Implementation, Coding Quality and Functionality

Implementation

- Build ETL and streaming pipeline
- Process trip, driver, and payment data
- Analyze peak hours and driver performance
- Predict demand using historical data
- Display results on Power BI

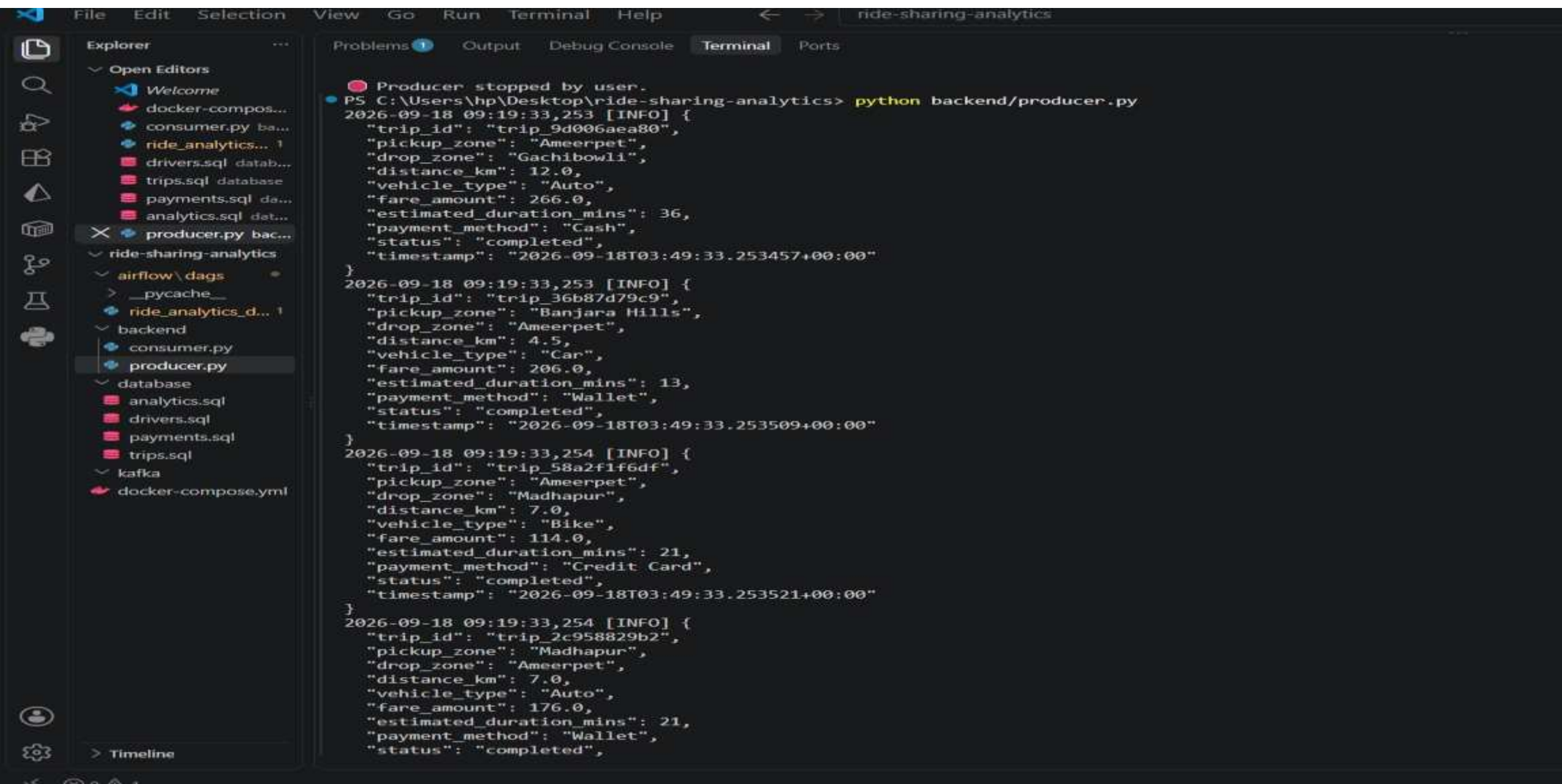
Coding Quality

- Modular Python code (separate extract/transform/load functions)
- Clear variable/function naming
- Data validation before storage
- Independently testable components

Functionality → (insert a similar small flow image: Trip event → Kafka → Airflow → PostgreSQL → Analysis → Power BI, instead of the text sentence)

- Real-time trip/driver data collection
- Peak zone and hour detection
- Driver performance tracking
- Better fleet planning

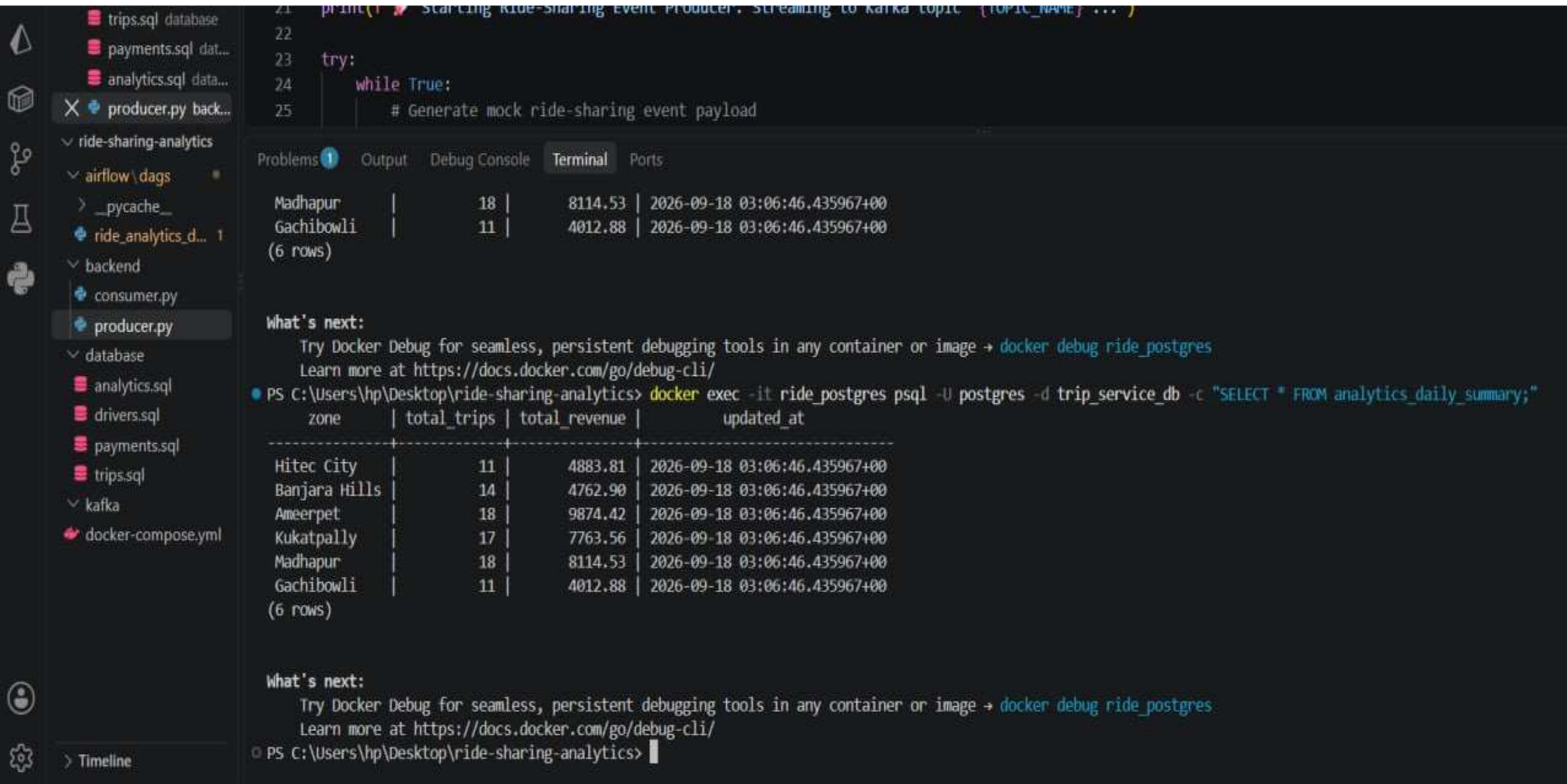
Implementation



The screenshot shows a Visual Studio Code editor with a terminal window open. The terminal displays the output of a Python script named `producer.py` located in the `backend` directory. The script has been executed, and the terminal shows a series of JSON objects representing trip data. The first object is a trip with ID `trip_9d006aea80`, pickup zone `Ameerpet`, drop zone `Gachibowli`, distance `12.0` km, vehicle type `Auto`, fare amount `266.0`, estimated duration `36` mins, payment method `Cash`, status `completed`, and timestamp `2026-09-18T03:49:33.253457+00:00`. The second object is a trip with ID `trip_36b87d79c9`, pickup zone `Banjara Hills`, drop zone `Ameerpet`, distance `4.5` km, vehicle type `Car`, fare amount `206.0`, estimated duration `13` mins, payment method `Wallet`, status `completed`, and timestamp `2026-09-18T03:49:33.253509+00:00`. The third object is a trip with ID `trip_58a2f1f6df`, pickup zone `Ameerpet`, drop zone `Madhapur`, distance `7.0` km, vehicle type `Bike`, fare amount `114.0`, estimated duration `21` mins, payment method `Credit Card`, status `completed`, and timestamp `2026-09-18T03:49:33.253521+00:00`. The fourth object is a trip with ID `trip_2c958829b2`, pickup zone `Madhapur`, drop zone `Ameerpet`, distance `7.0` km, vehicle type `Auto`, fare amount `176.0`, estimated duration `21` mins, payment method `Wallet`, status `completed`, and timestamp `2026-09-18T03:49:33.253521+00:00`. The terminal also shows a message: `Producer stopped by user.`

```
PS C:\Users\hp\Desktop\ride-sharing-analytics> python backend/producer.py
2026-09-18 09:19:33,253 [INFO] {
  "trip_id": "trip_9d006aea80",
  "pickup_zone": "Ameerpet",
  "drop_zone": "Gachibowli",
  "distance_km": 12.0,
  "vehicle_type": "Auto",
  "fare_amount": 266.0,
  "estimated_duration_mins": 36,
  "payment_method": "Cash",
  "status": "completed",
  "timestamp": "2026-09-18T03:49:33.253457+00:00"
}
2026-09-18 09:19:33,253 [INFO] {
  "trip_id": "trip_36b87d79c9",
  "pickup_zone": "Banjara Hills",
  "drop_zone": "Ameerpet",
  "distance_km": 4.5,
  "vehicle_type": "Car",
  "fare_amount": 206.0,
  "estimated_duration_mins": 13,
  "payment_method": "Wallet",
  "status": "completed",
  "timestamp": "2026-09-18T03:49:33.253509+00:00"
}
2026-09-18 09:19:33,254 [INFO] {
  "trip_id": "trip_58a2f1f6df",
  "pickup_zone": "Ameerpet",
  "drop_zone": "Madhapur",
  "distance_km": 7.0,
  "vehicle_type": "Bike",
  "fare_amount": 114.0,
  "estimated_duration_mins": 21,
  "payment_method": "Credit Card",
  "status": "completed",
  "timestamp": "2026-09-18T03:49:33.253521+00:00"
}
2026-09-18 09:19:33,254 [INFO] {
  "trip_id": "trip_2c958829b2",
  "pickup_zone": "Madhapur",
  "drop_zone": "Ameerpet",
  "distance_km": 7.0,
  "vehicle_type": "Auto",
  "fare_amount": 176.0,
  "estimated_duration_mins": 21,
  "payment_method": "Wallet",
  "status": "completed",
  "timestamp": "2026-09-18T03:49:33.253521+00:00"
}
Producer stopped by user.
```

Implementation



The screenshot shows a VS Code editor with a file explorer on the left and a terminal window at the bottom. The file explorer shows a project structure for 'ride-sharing-analytics' with files like 'trips.sql', 'payments.sql', 'analytics.sql', 'producer.py', 'consumer.py', and 'docker-compose.yml'. The terminal window shows the output of a Docker command to execute a PostgreSQL query.

Code Snippet (producer.py):

```
21 print(f"Starting Ride-Sharing Event Producer. Streaming to kafka topic {TOPIC_NAME} ...")
22
23 try:
24     while True:
25         # Generate mock ride-sharing event payload
```

Terminal Output:

Problems 1 Output Debug Console Terminal Ports

zone	total_trips	total_revenue	updated_at
Madhapur	18	8114.53	2026-09-18 03:06:46.435967+00
Gachibowli	11	4012.88	2026-09-18 03:06:46.435967+00

(6 rows)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → [docker debug ride_postgres](#)
Learn more at <https://docs.docker.com/go/debug-cli/>

● PS C:\Users\hp\Desktop\ride-sharing-analytics> **docker** exec -it ride_postgres psql -U postgres -d trip_service_db -c "SELECT * FROM analytics_daily_summary;"

zone	total_trips	total_revenue	updated_at
Hitec City	11	4883.81	2026-09-18 03:06:46.435967+00
Banjara Hills	14	4762.90	2026-09-18 03:06:46.435967+00
Ameerpet	18	9874.42	2026-09-18 03:06:46.435967+00
Kukatpally	17	7763.56	2026-09-18 03:06:46.435967+00
Madhapur	18	8114.53	2026-09-18 03:06:46.435967+00
Gachibowli	11	4012.88	2026-09-18 03:06:46.435967+00

(6 rows)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → [docker debug ride_postgres](#)
Learn more at <https://docs.docker.com/go/debug-cli/>

○ PS C:\Users\hp\Desktop\ride-sharing-analytics>

Implementation

 Airflow

DAGsDatasetsSecurityBrowseAdminDocs

03:28 UTC 

 DAG: ride_sharing_analytics_etl Hourly ETL pipeline for ride sharing data warehouse

Schedule: 1:00:00Next Run: 2026-09-17, 17:16:24

 Grid Graph Calendar Task Duration Task Tries Landing Times Gantt Details Code Audit Log

09/17/2026 05:16:57 PM  25  All Run Types  All Run States  Clear Filters

Auto-refresh 

deferredfailedqueuedremovedrestartingrunningscheduledshutdownskippedsuccessup_for_rescheduleup_for_retryupstream_failedno_status

aggregate Ride Data

Duration

00:00:03

00:00:01

00:00:00

09/18/2026 02:53

DAG: ride_sharing_analytics_etl

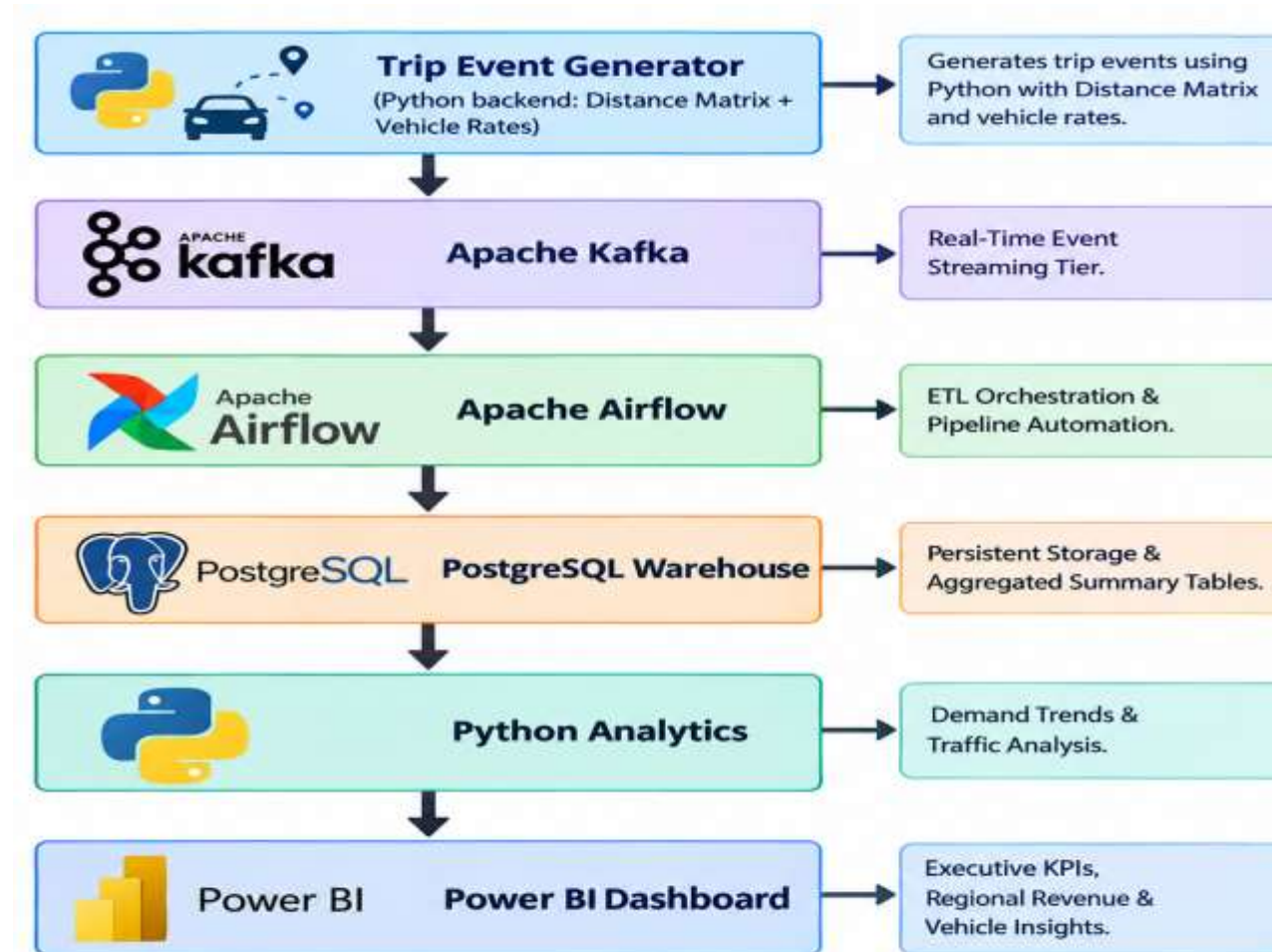
Run 2026-09-18, 02:53:47 UTC

ClearMark state as...

DetailsGraph

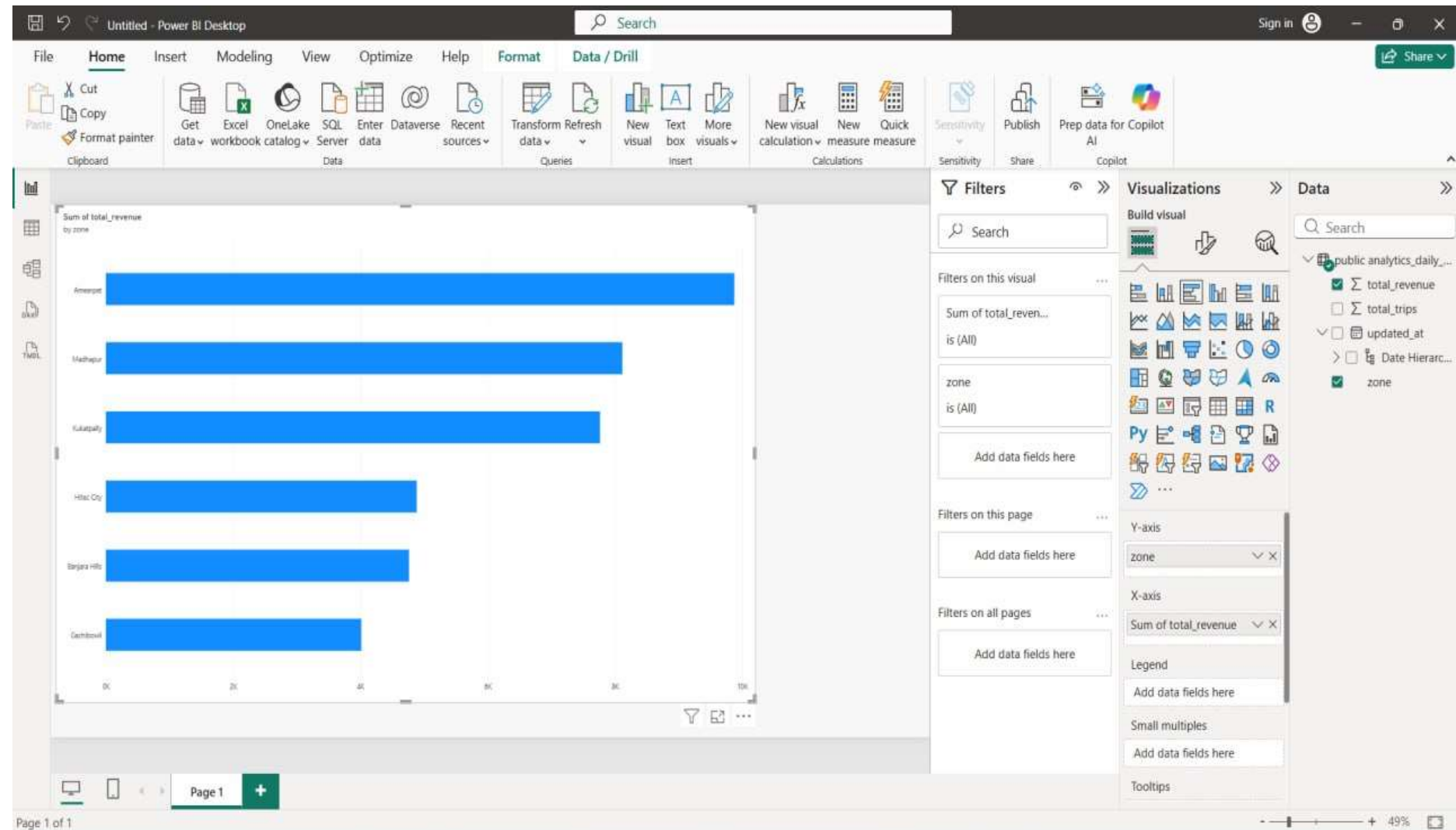
Run duration	00:00:02
Last scheduling decision	2026-09-18, 03:06:46 UTC
Queued at	2026-09-18, 03:06:43 UTC
Started	2026-09-18, 03:06:44 UTC
Ended	2026-09-18, 03:06:46 UTC
Data interval start	2026-09-18, 01:53:47 UTC
Data interval end	2026-09-18, 02:53:47 UTC
Externally triggered	False
Run config	None

Functionality / Flow Diagram



Results

- Successfully processes ride-sharing trip data.
- Identifies peak traffic hours across different zones.
- Provides driver performance insights.
- Analyzes ride-demand patterns.
- Generates future demand predictions using historical data.
- Provides interactive analytical dashboards.
- Supports better fleet planning and data-driven decision-making.



Comparison

Feature	Existing System	Proposed System
Data Processing	Mostly historical/batch analysis	Automated processing pipeline
Real-Time Streaming	Limited integration	Apache Kafka
ETL Automation	Limited/manual	Apache Airflow
Data Storage	Multiple/separate systems	PostgreSQL
Demand Analysis	Historical analysis	Historical + prediction
Driver Analysis	Basic monitoring	Performance analytics
Visualization	Limited/varied	Power BI dashboard
Deployment	Traditional deployment	Docker-based deployment
Decision Support	Delayed insights	Data-driven insights

Conclusion

- The **Ride-Sharing Analytics Platform** provides a structured approach to processing and analyzing ride-sharing data.
- Kafka enables real-time trip-event streaming.
- Airflow automates ETL workflows.
- PostgreSQL provides structured data storage.
- Python supports data processing and demand prediction.
- Power BI provides interactive visualization of analytical insights.
- Docker supports consistent and scalable deployment.
- The platform helps generate insights into **traffic, demand, and driver performance**.

References

- Use the papers already present in your Literature Survey:
- **Short-term prediction of demand for ride-hailing services: A deep learning approach (UberNet)** — Long Chen et al.
- **Analysis and prediction of New York City taxi and Uber demands**
- **Ride-hailing fares and demand interactions**
- **Predicting Rideshare Demand** — Ken Steif
- **Modeling and Prediction of Ride-Sharing Utilization Dynamics**
- **Real-time Text Analytics Pipeline Using Open-Source Big Data Tools**